Urumi FDE — Full Implementation Guide

Oct 9, 2026 · @Prithvi Ahuja

Architecture Overview

Five runtime layers talk to each other in one direction: the store sits at the bottom, a WooCommerce MCP server wraps it, the Gateway aggregates all upstreams, the Dashboard and Chat sit above, and a Create Store Service runs alongside to provision stores on demand.

Data stores

Store	What lives there	Why separate
Platform DB (Postgres)	Users, hashed API keys, short_chat_tokens, upstream configs with encrypted creds, team memberships	Fast transactional reads; never mixed with log volume
MCP Logs DB (Postgres)	Audit log: user, server, tool, timestamp, duration, outcome	Write-heavy; separated so log bursts never slow the platform
Per-store MySQL	WordPress + WooCommerce data, one instance per store	Isolates stores; makes multi-store provisioning and teardown clean
Redis	short_chat_tokens (TTL 30 min), upstream health-check state cache	Ephemeral; fast TTL lookups

Components

Component	Key responsibilities
MCP Gateway	API key auth, short_chat_token verify, Streamable HTTP, upstream registry, tool name prefixing (`slug__tool`), failure isolation, per-request timeout
WooCommerce MCP Server	7 tools over Streamable HTTP; calls WooCommerce REST API; no ingress outside cluster

Component	Key responsibilities
LiteLLM	One API for any LLM provider; streaming, fallbacks, retries, per-request timeouts; keys from Kubernetes Secrets
Chat Agent backend	Issues short_chat_token on session start; calls LiteLLM with tool schema from gateway; attributes all calls to logged-in user
Dashboard (React)	Admin: servers, members, keys, audit log, store list. Member: chat, own API key
Create Store Service	Helm-provisions new namespace + MySQL + WordPress + WooCommerce MCP server; registers in gateway; tears down cleanly on delete

External MCP client request flow

1. Client sends `Authorization: Bearer <api_key>` to gateway `/mcp`
2. Gateway hashes key, looks up user in Platform DB, checks role and revocation status
3. Gateway fans out `tools/list` to all enabled upstreams (result cached 60 s); prefixes names `<slug>__<tool>`
4. On `tools/call`, gateway routes to the right upstream, injects upstream credential from memory, records start time
5. Response returned; audit record written to MCP Logs DB with user, tool, duration, outcome

Chat agent request flow

1. User opens chat; browser calls `/chat/session` with session cookie
2. Backend generates short_chat_token (UUID, stored in Redis with 30 min TTL)
3. Chat backend calls LiteLLM with tool schema; LiteLLM calls gateway using short_chat_token as bearer
4. Gateway verifies token via Redis, resolves to user; audit log carries that user's id
5. LiteLLM streams response through backend to browser

Issues and Solutions

1. WordPress / WooCommerce Initialization on Kubernetes

Problem: WooCommerce is not self-configuring. A raw WordPress image starts with no database, no plugins, and no REST API keys. Your Create Store Service needs to handle this without manual steps.

Solution: A Kubernetes `Job` (Helm hook `post-install`) runs `wp-cli` inside the WordPress pod after it is ready:

```
1. wp core install --url --title --admin_user --admin_password --admin_email
2. wp plugin install woocommerce --activate
3. wp wc tool run install_pages --user=admin
4. wp wc payment_gateway update cod --enabled=true --user=admin
5. wp eval 'echo json_encode(wc_create_api_key(...));'
   → writes consumer_key + consumer_secret into a Kubernetes Secret
```

The WooCommerce MCP server reads that Secret at startup as `WOOCOMMERCE_KEY` and `WOOCOMMERCE_SECRET`. The Job is idempotent (wp-cli checks if already installed).

---

2. Tool Name Collision Across Upstream Servers

Problem: Two servers can expose `list_orders`. The MCP spec does not define how to handle this.

Solution: At registration time, every server gets a slug (e.g., `woocommerce`, `github`). The gateway prefixes every tool name on `tools/list`: `woocommerce__list_orders`, `github__list_repos`. On `tools/call`, the gateway strips the prefix to find the upstream. Store the slug in the upstream config table. Document this scheme in the system design note.

Edge case: if a slug itself contains `__`, reject it at registration time with a clear error.

---

3. Upstream Health-Check Loop

Problem: The dashboard must show Connected / Unreachable / Disabled. You cannot probe on every page load — it's too slow and creates thundering herd.

Solution: A background goroutine (or async task) in the gateway checks each enabled upstream every 30 seconds by calling `tools/list` with a 5 s timeout. Result written to Redis (`upstream:<id>:status` , TTL 60 s). Dashboard backend reads from Redis — instant response, no live probe. If an upstream is dead, only its tools disappear from the aggregated list; other upstreams are unaffected.

---

4. Upstream Credential Encryption at Rest

Problem: Upstream credentials stored in Platform DB must never be readable even if the DB is dumped.

Solution: Encrypt with AES-256-GCM. The encryption key lives in a Kubernetes Secret (`gateway-encryption-key`), never in the DB. On write: `encrypt(cred, key) → ciphertext` stored in `upstream_servers.credential_ciphertext` . On read: decrypt in memory, use for the upstream call, never return in any API response. The gateway reads the key from the env at startup.

---

5. WooCommerce MCP Server Must Not Be Reachable Externally

Problem: The spec requires only the gateway to call it. Kubernetes Services are reachable cluster-wide by default.

Solution: Two layers:

- No `Ingress` resource for the WooCommerce MCP server.
- A `NetworkPolicy` on the WooCommerce MCP pod that allows ingress only from pods with label `app: mcp-gateway` and denies everything else. Apply the same policy to per-store MCP servers created by the Create Store Service.

---

6. short_chat_token Security

Problem: The chat backend must call the gateway as the logged-in user so audit logs are attributed correctly. Passing the user's real API key to the browser is risky.

Solution (your design — confirmed good): On `/chat/session` , generate a UUID short_chat_token. Store it in Redis with a 30-minute TTL and in Platform DB (hashed, linked to user id). The browser never sees it — the backend holds it server-side. The chat backend injects it as the bearer token when calling LiteLLM → gateway. Gateway verifies via Redis. On token expiry, the next chat message triggers a silent refresh. Revoke on logout.

---

7. Slow or Dead Upstream Must Not Block Others

Problem: A single unresponsive upstream makes the whole `tools/list` hang.

Solution: Fan out `tools/list` to all upstreams concurrently using goroutines (Go) or `asyncio.gather` (Python). Per-upstream timeout: 5 s for listing, 30 s for tool calls. If one times out, log it as unreachable and return an error tool result for calls to it; the rest of the aggregated list is unaffected. Use `context.WithTimeout` in Go or `asyncio.wait_for` in Python.

---

8. Ingress and Local Domain Strategy

Problem: Minikube/Kind Ingress doesn't auto-resolve local hostnames.

Solution (local):

1. Enable the NGINX Ingress controller addon (`minikube addons enable ingress` or Kind's ingress config)
2. Get the Ingress controller IP (`kubectl get svc -n ingress-nginx`)
3. Add entries to `/etc/hosts` :

```
<ingress-ip>  store.local dashboard.local
```

4. `values-local.yaml` sets `ingress.host: store.local` and `ingress.host: dashboard.local`
5. `values-prod.yaml` sets real domains under `shikharsaxena.space` on AWS

Helm values control host names, storage class (`standard` local vs `gp2` AWS), and TLS (`false` local, `cert-manager` prod).

---

9. Three Required Upstream Servers

Problem: The spec requires at least three. Only WooCommerce was shown.

Solution: Add two more in-cluster servers:

- **GitHub MCP server** (use your personal token as the upstream credential stored encrypted in Platform DB) — exposes `list_repos` , `create_issue` , `get_pull_request` . This gives you the cross-server demo (`run a prompt that needs tools from two different upstreams`).

- **A simple custom MCP server** (e.g., a weather or currency API wrapper, ~100 lines of Node/Python) — trivial to write, gives you full control for the name-collision and tool-permission demos.

---

10. Store Provisioning Recovery (Idempotency)

Problem: If the Create Store Service crashes mid-provisioning, the store must either complete or fail cleanly on retry — no orphaned namespaces.

Solution: Store provisioning state in Platform DB as a state machine: `pending → provisioning → ready / failed`. On restart, scan for stores stuck in `provisioning` and retry or mark failed. Use Helm's `--atomic` flag so a failed install rolls back automatically. The `wp-cli` Job is idempotent (checks if installed). Write a finalizer pattern: the store record is not deleted from the DB until the namespace is confirmed gone.

Implementation Timeline

Deadline: 26 Oct. Core must work end-to-end before any bonus. Stop adding features when the definition-of-done flow is not yet passing.

Oct 9–11 — Database and Platform Backend

- Set up the monorepo: `/gateway`, `/dashboard`, `/woo-mcp`, `/helm`, `/create-store-service`
- Stand up Postgres (Platform DB + MCP Logs DB as two schemas or two instances in one StatefulSet)
- Stand up Redis
- Write the Platform DB schema: `users`, `api_keys` (hashed), `upstream_servers` (encrypted creds), `team_memberships`, `short_chat_tokens`
- Write the MCP Logs schema: `audit_log`
- Build auth endpoints: `POST /auth/register`, `POST /auth/login`, `POST /users/:id/api-keys`, `DELETE /users/:id/api-keys/:keyId`
- Build upstream server CRUD: `GET/POST/PUT/DELETE /upstream-servers`
- Write AES-256-GCM encrypt/decrypt utility; wire into upstream server save/load
- **Checkpoint:** Can create a user, issue an API key, add an upstream server config with encrypted creds

Oct 12–13 — MCP Gateway Core

- Implement Streamable HTTP server (`/mcp` endpoint)
- Implement API key authentication middleware (hash incoming key, look up in DB)
- Implement short_chat_token verification middleware (Redis lookup)
- Implement upstream registry: load enabled upstreams from DB on startup + on-change
- Implement `tools/list` with concurrent fan-out (5 s timeout per upstream) and `slug__tool` prefixing
- Implement `tools/call` routing: strip prefix, find upstream, inject credential, 30 s timeout
- Write audit log entry on every tool call (user, server, tool, timestamp, duration, outcome)
- **Checkpoint:** Gateway running locally; can call a mock upstream through it and see audit log entry

Oct 14 — WooCommerce MCP Server

- Stand up WordPress + WooCommerce in Docker (local, not k8s yet)
- Write the MCP server (Node.js or Python): Streamable HTTP, 7 tools calling WooCommerce REST API
  - `woocommerce__list_products` , `woocommerce__get_product` , `woocommerce__create_product` , `woocommerce__update_product`
  - `woocommerce__list_orders` , `woocommerce__get_order` , `woocommerce__update_order_status`
- Wire in env vars `WOOCOMMERCE_URL` , `WOOCOMMERCE_KEY` , `WOOCOMMERCE_SECRET`
- **Checkpoint:** Can call `list_products` through the gateway, see result, see audit log

Oct 15 — Chat Agent + LiteLLM + Dashboard Shell

- Stand up LiteLLM proxy with one provider (Anthropic free tier or Groq); configure from env
- Build `/chat/session` endpoint: issues short_chat_token, stores in Redis + DB
- Build `/chat/message` endpoint: calls LiteLLM with tool schema from gateway, streams response
- Build the React dashboard shell: login page, sidebar nav (Servers, Members, Audit Log, Chat)

- Wire the Chat page to `/chat/session` + `/chat/message` with streaming
- **Checkpoint:** Can type a question in the dashboard chat, agent calls a WooCommerce tool, response streams back

Oct 16 — Buffer + Integration Smoke Test

- Run the full definition-of-done flow manually: place order on storefront, ask chat agent for today's orders
- Fix any broken wiring between components
- Add the second and third upstream servers (GitHub MCP + custom server); test multi-server tool listing
- Test API key revocation and upstream disable flows

Oct 17–18 — Kubernetes: Core Cluster Setup

- Write Helm chart for Platform DB (Postgres StatefulSet, PVC, Service)
- Write Helm chart for Redis (StatefulSet, Service)
- Write Helm chart for MCP Gateway (Deployment, Service, ConfigMap for upstream list seed)
- Write Helm chart for Dashboard (Deployment, Service)
- Write Helm chart for WordPress + WooCommerce (StatefulSet for MySQL PVC, Deployment for WordPress, Services)
- Add `values-local.yaml` (Kind/Minikube storage class `standard`, local image pull policy)
- Add readiness (`/health`) and liveness probes to every Deployment
- Add resource requests and limits to every container
- Move all secrets to Kubernetes Secrets; remove hardcoded values from images
- **Checkpoint:** All components running in local Kubernetes; gateway reachable in-cluster

Oct 19–20 — Ingress + WordPress Init Job

- Configure NGINX Ingress controller (Kind or Minikube addon)
- Write Ingress resources for storefront (`store.local`) and dashboard (`dashboard.local`)
- Write `/etc/hosts` instructions for local domain setup
- Write `wp-cli` Kubernetes Job (Helm hook `post-install`) for WordPress init

- Test the full flow over Ingress: browser to storefront, browser to dashboard, external MCP client to gateway
- Write `values-prod.yaml` (real domain under `shikharsaxena.space`, AWS storage class `gp2`, cert-manager TLS)
- **Checkpoint:** Definition-of-done flow working end-to-end over Ingress with two different users

Oct 21 — Upstream Health Check + Audit Log UI

- Implement background health-check loop in the gateway (30 s interval, Redis cache)
- Build the Servers page in the dashboard: show Connected / Unreachable / Disabled per upstream
- Kill a pod; confirm it shows Unreachable within 60 s; confirm other servers still work
- Build the Audit Log page in the dashboard: table with user, server, tool, timestamp, duration, outcome
- **Checkpoint:** Kill-pod scenario and audit log both working

Oct 22–23 — Bonus: Multi-Store Provisioning

- Build Create Store Service: Helm SDK call to install a new store chart in its own namespace
- Implement provisioning state machine in Platform DB (`pending → provisioning → ready / failed`)
- Auto-register new store's MCP server in gateway with slug `store-<id>__`
- Build Stores page in dashboard: create, delete, status display
- Write NetworkPolicies: deny-by-default, allow only gateway → MCP server in each namespace
- Test clean teardown: delete store, confirm namespace gone, tools gone from gateway
- **Checkpoint:** Multi-store provisioning working end-to-end

Oct 24 — Extra Standout Items

- Implement per-role tool permissions (admin can write, member read-only; restricted tools hidden from list)
- Implement prompt injection detection on tool descriptions (flag at registration, show warning badge)
- Add tool usage heatmap to the dashboard Audit Log page

- Write the threat model note
- Write the system design and tradeoffs note

Oct 25–26 — Polish, Docs, Video

- Write README: local setup, VPS setup, how to place an order, connect external MCP client, use chat
- Record the demo video covering all sections in the spec: system design, Kubernetes, MCP internals, security, chat agent, local-to-VPS story
- Final smoke test: full definition-of-done flow with two users, both in the video
- Submit via the Google Form

Extra Touches That Will Impress

Each item below is small to build (2–4 hours each) but highly visible in the demo. Pick depth over breadth — two done well beat five half-done.

---

1. Prompt Injection Detection on Tool Descriptions (Highest impact)

When an admin registers an upstream MCP server, the gateway scans every tool's `description` field for known attack patterns:

- Keywords: `ignore previous instructions`, `you are now`, `disregard`, `system prompt`, `forget`
- Abnormal length: description > 500 characters (a common hiding spot for injected instructions)
- Self-referential instructions: description that tells the model what to call next

If any tool description matches, the server gets a warning badge in the dashboard: `"Suspicious tool descriptions detected — review before enabling"`. Admins can still enable it but must acknowledge the warning.

Why it wins: The spec's threat model section explicitly asks *"what happens if an admin adds a malicious or compromised MCP server?"* Every other candidate answers this in a doc. You will have built the answer into the UI. This directly shows you read and understood the security requirements, not just the happy path.

---

2. short_chat_token as a Design Decision (Already planned — make it visible)

Your short_chat_token design is genuinely unusual and interviewers will ask about it. Make it easy to explain:

- Show it in the dashboard: a "Sessions" row under the user's profile showing active chat tokens and their expiry
- Log token issuance and expiry in the audit log as system events
- In your demo video, open Redis CLI and show the token TTL ticking down

This turns an invisible backend decision into a demonstrable security property.

---

3. Tool Usage Heatmap in the Audit Log (Visually memorable)

Above the audit log table, add a simple 7×24 heatmap (days × hours) showing call volume per cell. Built with a CSS grid or a tiny canvas element — no charting library needed.

What it shows at a glance: when your team uses the gateway most, which hours have the most errors (colour the error rate differently). Takes about 3 hours to build.

Why it wins: The demo video is visual. This is the one screenshot that will stand out in a reviewer's memory when they see 10 similar dashboards.

---

4. Per-Role Tool Permissions with Hidden Restricted Tools

Two tables in Platform DB: `role_tool_permissions` (role, upstream_slug, tool_name, allowed: bool). On `tools/list`, the gateway filters the tool list by the calling user's role. On `tools/call`, if the tool is not in the allowed list for that role, return a structured error: `{"error": "tool_not_permitted", "tool": "woocommerce__create_product"}`.

In the dashboard, show admins a "Permissions" tab per server where they can toggle which tools members can use.

Demo value: show that a member's `tools/list` does not include `create_product`, then try to call it directly with MCP Inspector and show the rejection.

---

5. Gateway Health Endpoint for Kubernetes + Dashboard

Expose `GET /health` on the gateway that returns:

```
{
  "status": "ok",
  "upstreams": [
    {"slug": "woocommerce", "status": "connected", "tools": 7, "latency_ms":
43},
    {"slug": "github", "status": "unreachable", "last_seen": "2026-10-
14T10:23:00Z"}
  ]
}
```

Used as the liveness/readiness probe AND the data source for the dashboard's server status cards. One endpoint serves both Kubernetes and the UI. Shows you understand that operational endpoints are not just for humans.

---

6. Confirmation Step for Write Tools in Chat

Your `functions.png` already shows "Confirmation for write tools" in the Chat Agent box. Make this explicit:

- Before the chat agent executes any tool that mutates state (`create_product`, `update_order_status`, `delete`), it pauses and shows the user: *"I'm about to call `woocommerce__create_product` with these arguments. Confirm?"*
- A confirm/cancel button. On cancel, the tool call is skipped and the agent explains what it would have done.

Why it matters for the interview: The spec asks about security posture. An agent that creates products without confirmation is a liability. This shows you thought about agent safety, not just agent capability.

---

7. What to Show in the Demo Video (Sequencing)

The video order that lands best:

1. Open dashboard — show server status cards with Connected / Unreachable badges (health endpoint powering it)
2. Show WooCommerce storefront — place an order
3. Ask chat agent for today's orders — show confirmation step, show tool call attribution in audit log
4. Connect MCP Inspector as user B — show tool list with prefixed names

5. Revoke user B's key — show next call rejected
6. Kill a pod — show dashboard flip to Unreachable within 60 s, other servers unaffected
7. Register a new server with suspicious tool descriptions — show warning badge (prompt injection detection)
8. Delete a store — show namespace gone, tools gone from gateway
9. Show audit log heatmap with all the calls you just made

This order tells a story. Each step builds on the last. The final heatmap is a visual payoff showing the full session's activity.